

KKO 2025/2026: Projekt

Téma: Komprese obrazových dat s využitím slovníkové metody komprese dat

Datum odevzdání: nejpozději do 10. 5. 2026 (včetně) prostřednictvím IS VUT.

Forma odevzdání: elektronicky - soubory v archivu ZIP

Počet bodů: max. 30

Poznámka: k zadání je v IS VUT zveřejněna sada testovacích obrázků a kostra implementace

Dotazy: vasicek@fit.vutbr.cz

Zadání:

Cílem projektu je vytvořit konzolovou aplikaci v C/C++ pro kompresi šedo tónových obrazových dat.

Podrobné pokyny k vypracování:

a) použitá kompresní metoda

Kompresi bude zajišťovat vhodně zvolená slovníková metoda (LZ77, LZ78, případně libovolná jiná i vlastní varianta). Volba metody je součástí řešení.

b) formát vstupních dat:

Vstupní data budou reprezentována ve formátu RAW (surová data bez hlavičky a bez použití komprese). Jednotlivé obrazové body (pixely) mohou nabývat hodnot odstínů šedi v intervalu 0 až 255. Z důvodu použití RAW formátu je tedy nutné aplikaci prostřednictvím parametru v příkazové řádce definovat rozměry vstupního obrazu (šířku). Pro zjednodušení implementace je možné počítat s tím, že šířka i výška bude vždy násobek 256.

c) zpracování vstupních dat:

Serializace dat: Implementujte statickou a adaptivní metodu skenování vstupního obrazu (viz [2], skenování obrazových dat v části věnující se RLE). V případě statické verze procházejte vstupní obraz horizontálně a celý vstup chápejte jako jeden souvislý blok dat. U adaptivní verze uvažujte minimálně dva typy průchodu: horizontální a vertikální. Je také výhodné obraz rozložit na menší bloky – pro jednoduchost stačí implementovat pouze bloky fixní velikosti – např. 16x16 pixelů (max 256x256). Typ průchodu volte v případě adaptivní verze tak, aby bylo dosaženo co nejvyšší kompresní účinnosti. Pokud se nepodaří dosáhnout komprese, uložte data v původní nekomprimované podobě. Každý blok nebo skupina bloků si tedy bude muset nést ve zkomprimovaných datech pomocnou informaci o způsobu zpracování bloku (zvolená varianta skenování, použití komprese).

Transformace dat: Na základě parametru v příkazovém řádku pracujte se serializovanými vstupními obrazovými daty dvěma různými způsoby. Buď berte v potaz *přímo hodnotu samotných pixelů* nebo použijte vhodný *model umožňující zvýšit kompresní účinnost*. Jako model lze použít např. difference sousedních pixelů (viz úvodní slidy KKO), případně jakýkoliv jiný pokročilejší kontextově orientovaný přístup.

d) funkce a uživatelské rozhraní:

Výsledné řešení bude mít charakter konzolové aplikace spustitelné z příkazového řádku pomocí příkazu *lz codec*, který získáme po překladu zdrojového kódu. Funkci aplikace bude možné řídit pomocí sady přepínačů či parametrů zadávaných na příkazovém řádku:

- c volba režimu, aplikace bude vstupní soubor komprimovat,
- d volba režimu, aplikace bude vstupní soubor dekomprimovat,
- m aktivuje model (viz bod c) zadání) pro předzpracování vstupních dat
- a aktivuje režim adaptivního skenování obrazu (viz bod c) zadání); jinak je použito statické sekvenční skenování v horizontálním směru bez dělení na bloky
- i <ifile> název vstupního souboru. Dle režimu činnosti aplikace, viz přepínače -c/-d výše, se jedná

buď o data určená ke kompresi nebo dekompresi.

-o <ofile> název výstupního souboru. Dle režimu činnosti aplikace, viz přepínače -c/-d výše, se jedná buď o zkomprimovaná či dekomprimovaná data,

-w <width_value> udává šířku obrazu, přičemž platí vztah $width_value \geq 1$. Parametr se zadává pouze v případě komprese, aby aplikace věděla, jak široký obraz je. Pro potřeby dekomprese je nutné uložit všechny potřebné informace do výstupního souboru.

e) komentáře zdrojových kódů:

Zdrojový kód vhodně komentujte. Komentujte zejména parametry a činnost funkcí, datové typy, lokální proměnné (pokud to nebude z názvu přímo vyplývat).

Každý zdrojový soubor opatřete hlavičkou, ve které bude datum vytvoření, název a stručný popis souboru a informace o autorovi – jméno, příjmení a login.

f) kompilátor a prostředí:

Odevzdané řešení musí být možné přeložit a spustit na serveru merlin.fit.vutbr.cz, kde bude probíhat ověření funkčnosti a měření výkonnosti aplikace. Součástí zadání je minimální kostra parsující argumenty a Makefile pro překlad a spuštění jednoduchých testů.

g) dokumentace:

Součástí odevzdaného řešení bude stručná dokumentace, která bude na začátku obsahovat stručný rozbor řešeného problému (není žádoucí opisovat dlouhé teoretické statě z odborné literatury či studijních materiálů). Dále nástin vlastního řešení včetně např. vysvětlení funkce důležitých datových struktur a dalších relevantních aspektů. Nezapomeňte v dokumentaci uvést i odkazy na použité informační zdroje, z nichž jste čerpali během řešení projektu, nejedná-li se o přednášky.

Povinnou součástí dokumentace je taktéž vyhodnocení efektivity v těchto režimech činnosti:

- Komprese využívající statické skenování obrazu, bez modelu
- Komprese využívající statické skenování obrazu, s modelem (tj. parametr -m)
- Komprese využívající adaptivní skenování obrazu, bez modelu (tj. parametr -a)
- Komprese využívající adaptivní skenování obrazu, s modelem (tj. parametry -a -m)

Efektivitu komprese vyjádřete coby průměrný počet bitů potřebný na zakódování jednoho pixelu obrazu a času komprese ve výše uvedených režimech činnosti nezávisle pro jednotlivé případy obrazových dat. Naměřená data prezentujte formou tabulky. Pro každý obraz spočítejte entropii zdroje, budeme-li uvažovat, že výskyty hodnot jsou izolované a navzájem nezávislé. Uveďte také průměrný počet bitů pro jednotlivé metody vypočtený přes všechna přiložená data.

Rozsah dokumentace by neměl přesáhnout 4 strany formátu A4.

h) odevzdává se (checklist):

Archív ve formátu ZIP, který bude mít následující obsah:

- všechny zdrojové soubory aplikace implementované v jazyce C/C++,
- Makefile soubor (zavoláním *make* musí vzniknout spustitelný soubor *lz_codec*),
- dokumentace ve formátu PDF.

Použití externích kompresních knihoven není dovoleno s výjimkou *libdivsufsort*. Použijete-li knihovnu, nezapomeňte ji umístit do odevzdávaného archivu. V Makefile je možné si vytvořit vlastní target, případně dodat další funkcionalitu. Při překladu však nevolejte kromě G++ další nástroje kvůli možnosti automatického překladu v rámci sandboxu.

i) bonusové body:

Prvních 5 řešitelů s nejefektivnější implementací z pohledu dosažené míry komprese a rychlosti komprese a dekomprese má možnost získat až 3 bonusové body.

Literatura:

[1] Salomon, D.: “Data Compression, The Complete Reference,” NY, USA, Springer, 2000

[2] Přednášky k předmětu Kódování a komprese dat

Tipy a rady na závěr:

- Důležitou vlastností je přenositelnost. Pro různé platformy může být velikost typů `int`, `long` různá. Těmto problémům se vyhneme použitím předdefinovaných typů `int8_t`, `int16_t`, `int32_t`, `uint8_t` apod., které jsou definovány v knihovně `stdint.h`.
- Pro implementaci některých datových struktur a operací s nimi může být výhodné použít v aplikaci knihovnu STL (C++ Standard Template Library).
- Pro ladění potíží s pamětí (Segmentation fault apod.) doporučuji použít nástroj *valgrind*, který se používá na aplikaci zkompilevanou pomocí *make debug* (příp. *gcc -ggdb3*).